

Code Challenge: Senior Platform Engineer

About

The Basics

Spend as much time as you'd like on this challenge. We value quality and thoughtful decisions over completeness; if you don't finish everything, that's okay! We're looking for clean code, insight into your skills and strengths, and your ability to implement cloud platform and infrastructure best practices.

Setup

You'll need to have access to an AWS account to complete this coding exercise. You can sign up for a free account [here](#).

Please push your code to a GitHub repository that we can access (read-only) and try to maintain good Git practices.

Concepts

We want this to be fun! When you meet with our Cloud & Platform Engineers to review what you've built, it will be a discussion about your design decisions, why you chose what you did, and how it could be improved. The coding exercise requirements are meant to be high level and open ended so that you can choose how you want to show us your stuff.

We'll do our best to get back to you in a timely manner.

Our team values:

- Clean & concise code
- [Best practices](#) in code style and structure
- Simple architectures that are conducive to long term sustainability and maintenance
- Automation
- Observability (monitoring, logging, alerting, and all the telemetry)
- Immutable infrastructure
- Iterative value delivery (start with MVP and iterate to improve)
- Leverage AI in development workflows

Our current stack

- DevEx: GitHub, GitHub Actions, Claude Code, etc.

- Client & Services: JavaScript, TypeScript, Java, React.js, Node.js, Python, etc.
- Infrastructure: Terraform, everything AWS (ECS, Lambda, CloudFront, S3, ElastiCache, RDS, Systems Manager, etc.)

For CI/CD and your public repo, any public source control with a visible CI workflow file is acceptable. For example, GitLab for repository and CI/CD is acceptable. Bonus points for showing us your skills in the above technologies.

What We're Looking For

Area	Weight	What we evaluate
Automation service	30%	Code quality, error handling, architecture decisions, developer experience
Infrastructure design & Terraform quality	25%	Module structure, testing, idiomatic HCL, security posture
AI-native development workflow	20%	How effectively AI was used as a development collaborator, not just mentioned
CI/CD & operational maturity	15%	Pipeline design, observability, deployment strategy
Communication & documentation	10%	README clarity, design rationale, diagram quality, commit history

Your Mission

You can choose your own challenge! We have a core challenge that is mandatory, and you may choose one or more additional challenges to prove your areas of expertise. Complete the core challenge AND at least one other challenge.

Core Challenge

You are a Platform Engineer who designs and builds Internal Developer Platforms (IDPs) that enable software developers to self-serve infrastructure, testing, and deployment tools. We expect you to have a strong AI development workflow.

1. AI-Native Development Workflow

We expect you to use AI tools (Claude Code, Copilot, Cursor, or similar) as a core part of your development workflow for this challenge.

Requirements:

- Include your AI agent configuration in the repo (e.g., `CLAUDE.md`, custom instructions, MCP configs, or equivalent for your tool of choice)
- Leave at least one pull request open that shows your AI-assisted development process (commits, conversation, iteration)
- Be prepared to discuss: what worked well, where you had to course-correct the AI, and how you'd improve the workflow

2. Terraform & CI/CD

Using Terraform, deploy all workloads via a CI/CD pipeline of your choice.

Terraform guidelines:

- Include reusable modules as needed
- Include [Terraform Tests](#) in one or more modules that verify the required configuration
- Include a CI/CD pipeline for deploying Terraform resources
- Follow least-privilege IAM principles; no wildcard policies
- Use SSM Parameter Store, Secrets Manager, or similar for any sensitive values (no hardcoded secrets)

3. Automation Service (Primary Focus; be creative!)

Build a service (using AWS ECS Fargate or Lambda) that supports a common platform or developer workflow. Think "platform-as-a-product": who is the user of this service, and what's their experience like?

Use a language of your choice (Python, TypeScript, Go, etc.). Examples to spark ideas (you are not limited to these):

- A self-service API that provisions or configures cloud resources on behalf of development teams

- A webhook handler that reacts to GitHub/GitLab events (e.g., auto-labels pull requests, enforces branch naming conventions, posts deployment summaries via SNS)
- A CLI or API that aggregates health checks across multiple services
- A developer portal backend that catalogs services, environments, or infrastructure

Requirements for the service:

- Include input validation and robust error handling
- Be containerized (using a Dockerfile) or packaged for AWS Lambda
- Be exposed via a load balancer, REST API Gateway, or CloudFront
- Be accessible during the technical interview to demonstrate functionality
- Demonstrate observability practices. For example: structured logging, CloudWatch alarms with SNS notifications for key service metrics (CPU/memory for ECS, invocation count/error rates for Lambda), or a simple dashboard. We're looking for operational awareness, not checkbox coverage.
- The service should be able to record important data into one of the following (reports, API outputs, or files generated by the service):
 - An S3 bucket
 - A DynamoDB table
 - An RDS Aurora Postgres instance/cluster

4. Documentation

Requirements:

- A README with steps to deploy what you've built and an explanation of what you built
- A short design rationale (1–2 paragraphs): explain a key design decision you made and what alternatives you considered. This can be in the README or a separate `DECISIONS.md`.
- A technical diagram using draw.io, or similar, to showcase a simple architectural diagram. Commit the image and/or raw file (`.xml` format) into the repo in a `diagrams/` directory.

Digging Deeper

Choose at least one of these topics to supplement the core challenge. Go as deep as you want; depth in one area beats surface coverage of many. You are encouraged to tie them into other parts of the challenge to showcase a comprehensive design.

All ideas listed in each option are suggestions, not requirements. You are free to be creative.

Option 1: More Complex Terraform

Make everything a module. Enforce encryption at rest with customer-managed KMS keys on everything compatible. Enforce encryption in transit. Log everything. Create a serverless RDS cluster that your application queries. Set up autoscaling for your service(s). Demonstrate automated checks, validation, and IaC-related tools in your CI/CD pipeline.

Option 2: Show Off AI Maturity

Use Bedrock in your containerized service. Demo safe AI usage. Have multiple AI tasks work together. Use AI for your pull request reviews and leave a sample PR up (can include intentionally bad code to showcase).

Option 3: Show Off Dev Skills in Your App

Show off your coding skills by writing your own application. Any language mentioned above is welcome. Tie this into some data layer services you build out. Use IAM authentication for Postgres or another data layer service. Could write an API (language open: Python/FastAPI, Go/Gin, etc.). Add health checks in the service and infrastructure.

Option 4: Operational Intelligence

Build a tool that aggregates and presents platform operational data. Requirement: Use Python or Go. Examples: GitHub Actions execution history via the GitHub API; an audit of encryption and logging configuration across S3 buckets; IAM role usage reports showing which services roles have accessed within a specific period; CloudWatch Log Group ingestion reports; resource inventories grouped and sorted by type and name.

Option 5: More Detailed Diagram(s)

Rather than a simple diagram outlined in the core challenge, provide more detailed diagram(s). This could be a production-ready GitOps workflow or a secure, highly available infrastructure architecture.

Option 6: Something Else Cool

Whatever you think is relevant, unique, cool, and above all, valuable as a platform engineer. Can be a combination or variation of anything above.

The Follow Up

If we move forward, you'll virtually meet with the team to review your work and we will:

- Most importantly, get to know you!
- Be prepared to:

- Screen share and show your functioning solution
- Talk through your design and key decisions
- Talk through your code
- Baseline check: Complete a brief command-line or coding exercise without AI assistance to demonstrate foundational skills
- AI workflow demo: Show us how you work with AI tools in your normal development workflow - extend your service, debug an issue, or write a new module live
- Show us anything you have deployed or screenshots of deployment if you'd like

Good luck and we are looking forward to chatting with you!